

مسار واستدعاء الـ API خطوة بخطوة في تطبيق Movie App

شرح شامل لتدفق البيانات بحسب Clean Architecture من الواجهة إلى السيرفر والعودة

Movie App • Flutter BLoC / Cubit • Dio • Entity Mapping • Error Diagnosis

1. المخطط العام لدورة حياة الطلب (Request Lifecycle)

المسؤوليات إلى 3 طبقات رئيسية، بحيث تعتمد الطبقات الخارجية على الطبقات الداخلية (اتجاه التدفق من Presentation إلى Domain ثم Data):
تعتمد معمارية الـ Clean Architecture على فصل

الطبقة / المكون	الملف والمسؤولية
1. Presentation (UI Screen)	HomeScreen: تبدأ بطلب البيانات MovieCubit.fetchMovies() وتستمتع لتغيير الحالات (Loading, Error, Success).
2. Presentation (Cubit / State)	MovieCubit: يطلق MovieLoadingState ثم يستدعي GetAllMovies UseCase.
3. Domain (Use Case)	GetAllMovies: يمثل منطق العمل (Business Logic)، ينفذ استدعاء repo.getAllMovie().
4. Domain (Repository Contract)	MovieRepo: واجهة (Interface) مجردة تحدد عقد إرجاع <FutureResult>.
5. Data (Repository Imp)	MovieRepoImp: ينسق بين الـ RemoteDataSource ويحول Models إلى Entities عبر toEntity().
6. Data (Remote DataSource)	MovieRemoteDataSourceImpl: يرسل طلب HTTP عبر Dio إلى السيرفر الخارجي.
7. External API	(كاملة JSON، يرجع بيانات) https://movies-api.accel.li/api/v2/list_movies.json .

2. المسار التفصيلي بالكود خطوة بخطوة

الخطوة 1: انطلاق الطلب من الواجهة والـ Cubit

```
// HomeScreen.dart - ■■■■ Cubit ■■■■ ■■■■
BlocProvider(
  create: (context) {
    final remoteDataSource = MovieRemoteDataSourceImpl();
    final repo = MovieRepoImp(remoteDataSource);
    final getAllMoviesUseCase = GetAllMovies(repo);
    return MovieCubit(getAllMoviesUseCase: getAllMoviesUseCase)..fetchMovies();
  },
  child: HomeScreenContent(),
);
```

الخطوة 2: إدارة الحالة في الـ MovieCubit

```
// MovieCubit.dart - ■■■■ ■■■■ ■■■■ ■■■■
Future fetchMovies() async {
  emit(MovieLoadingState()); // 1. ■■■■ ■■■■ ■■■■
  final result = await getAllMoviesUseCase.call();
  result.fold(
    (failure) => emit(MovieErrorState(failure.message)), // ■■ ■■■■ ■■■■
    (movies) => emit(MovieSuccessState(movies)), // ■■ ■■■■ ■■■■
  );
}
```

الخطوة 3: الاتصال بالإنترنت وجلب البيانات عبر Dio

```
// movie_remote_data_source.dart - ■■■■■■■■ ■■■ Endpoint ■■■■■■■■ ■■■ JSON
final response = await DioService.dio.get('https://movies-api.accel.li/api/v2/list_movies.json');
if (response.statusCode == 200) {
  final Map data = response.data is String
  ? jsonDecode(response.data as String)
  : (response.data as Map);
  final movieModel = MovieModel.fromJson(data);
  return movieModel.data?.movies ?? [];
}
```

الخطوة 4: تحويل الـ Model إلى Entity (toEntity)

وظيفة الـ Repository هي تحويل كائنات الـ Data Model إلى MovieEntity النظيفة لحماية شاشات التطبيق من التبعيات الخارجية:

```
// movie_model.dart -> MoviesExtension
extension MoviesExtension on Movies {
  MovieEntity toEntity() {
    return MovieEntity(
      id: id ?? 0,
      title: title ?? '',
      image: mediumCoverImage ?? largeCoverImage ?? '',
      rating: rating ?? 0.0,
      year: year ?? 0,
      runtime: runtime ?? 0,
      summary: (summary != null && summary!.isNotEmpty) ? summary! : (descriptionFull ?? ''),
      genres: genres ?? const [],
      backgroundImage: backgroundImageOriginal ?? backgroundImage ?? '',
    );
  }
}
```

الخطوة 5: تمرير الفيلم إلى شاشة التفاصيل (MovieDetails)

عند النقر على أي فيلم في الشاشة الرئيسية يتم إرساله كـ argument واستقباله وعرض جميع تفاصيله:

```
// 1. ■■■■■■■■ ■■■ HomeScreen:
Navigator.pushNamed(context, RouteName.movieDetailsScreen, arguments: movie);

// 2. ■■■■■■■■ ■■■ MovieDetails:
final movie = ModalRoute.of(context)?.settings.arguments as MovieEntity?;

// 3. ■■■■■ ■■■ ■■■■■■■■:
MovieDetailsHeader(title: movie.title, year: movie.year.toString());
MovieStatBadge(icon: Icons.star, label: movie.rating.toStringAsFixed(1));
MovieStatBadge(icon: Icons.timer, label: '${movie.runtime}m');
Text(movie.summary);
```

3. دراسة الخطأ السابق (Exception: Unexpected Error) وكيف حل

في Dart، نوع int لا يتحول تلقائياً لـ double. عندما أرجع الـ API فيلم بتقييم 0 كـ int، تسبب سطر `rating = json['rating']` في رمي: السبب الجذري: `type 'int' is not a subtype of type 'double?' in type cast`
التحويل الآمن باستخدام `num`: الحل الجذري المعتمد:
`rating = (json['rating'] as num?).toDouble();`